# VAMOS: A Framework for Fast, Configurable and Accessible Multiobjective Optimization in Python

Nicolás R. Uribe, Alberto Herrán, Antonio J. Nebro, Javier Del Ser, and J. Manuel Colmenar

***Abstract*—Despite the interpreted nature of Python, which makes metaheuristics implemented in this language inherently slower than those of frameworks written in compiled languages, it remains a widely adopted platform for empirical studies in multi-objective optimization. Most existing Python libraries further aggravate this limitation by managing populations as collections of individual solution objects, preventing effective vectorization and keeping hot-path computations in the interpreter. In this paper, we introduce VAMOS (Vectorized Architecture for Multiobjective Optimization Studies), a Python framework that stores the population state in dense numerical arrays dispatching hot-path computations to pluggable backends and keeping the algorithmic logic independent of the numerical substrate. VAMOS implements eight multi-objective evolutionary algorithms (MOEAs) covering dominance, decomposition, and indicator-based paradigms, with a richer component-level configurability than its competitors. It also provides an automatic algorithm configuration tool, together with a browser-based graphical interface to enhance accessibility for non-expert users. The performance of VAMOS is compared to existing Python frameworks for multi-objective optimization on several benchmark suites. Paired Wilcoxon signed-rank tests with Holm correction on the aligned NSGA-II baseline confirm that VAMOS achieves statistically equivalent solution quality to state-of-the-art Python frameworks, with a reduction in wall-clock runtime.**

***Index Terms*—Evolutionary algorithms, multiobjective optimization, software framework, vectorization**

## I. Introduction

MULTI-OBJECTIVE optimization problems (MOPs) arise whenever trade-offs exist between conflicting objectives, requiring the computation of an accurate approximation of the Pareto-optimal set rather than a single optimum [1]. Such problems are common in engineering design, machine learning, scheduling, and resource allocation, among other domains. In this field, multi-objective evolutionary algorithms (MOEAs) remain a dominant approach due to their robustness, population-based anytime behavior, and ability to return a set of non-dominated solutions in a single run [2].

The widespread adoption of MOEAs has been fueled in large part by the availability of software frameworks that lower the implementation barrier and facilitate reproducible experimentation. Early efforts such as *PISA* [3] established the concept of modular, platform-independent interfaces for multi-objective search. The *jMetal* framework [4] later consolidated a comprehensive Java-based framework that became a reference for algorithm development and research studies. More recently, *PlatEMO* [5] has provided a MATLAB platform with a large catalog of algorithms and test problems. In parallel, the Python ecosystem has seen the emergence of dedicated libraries, such as *pymoo* [6], *jMetalPy* [7], *Platypus* [8], and *DEAP* [9], that leverage Python's accessibility and rich scientific stack.

As Python has become a widely adopted platform for MOEA research and experimentation, three practical limitations hinder its broader adoption. First, the interpreted nature of Python, combined with libraries that keep hot-path computations inside the interpreter [7]–[9], makes the inner loops of pure-Python implementations inherently slower than those of frameworks written in compiled languages such as C++ or Java. Second, existing MOEA frameworks often provide limited configurability at the component level and do not allow attaching external archives, useful to allow NSGA-II to handle problems with more than two objectives [10], with the exception of *jMetal* [4]. Third, most existing frameworks present a high entry barrier for non-expert users while simultaneously lacking the necessary flexibility for advanced researchers to achieve fine-grained algorithmic configuration. To overcome these limitations, we introduce VAMOS, a Python-based package designed with a single guiding principle: keep algorithm state in dense arrays and push hot loops into vectorized kernels. Specifically, the main contributions of VAMOS are summarized as follows:

a) *Performance*: A unified array-based internal representation with pluggable compute kernels that reduces inner-loop overhead relative to object-centric libraries.
b) *Configurability*: Eight MOEA implementations with high component-level configurability and an automatic algorithm configuration module.
c) *Accessibility*: An accessible API and tooling for novice-to-expert workflows.

The rest of the paper is organized as follows. Section II reviews the existing Python MOEA frameworks. Section III presents the VAMOS framework. Sections V, VI, and VII address the three contributions—performance, configurability, and accessibility—respectively. Finally, Section VIII draws conclusions and outlines directions for future work.

## II. Related Work

In this section, we review the most relevant Python libraries for MOEAs, namely *pymoo*, *jMetalPy*, *DEAP*, *Platypus*, and

N. R. Uribe, A. Herrán, and J. M. Colmenar are with the Department of Computer Sciences, Universidad Rey Juan Carlos, Móstoles, 28933 Madrid, Spain (e-mail: nicolas.rodriguez@urjc.es; alberto.herran@urjc.es; josemanuel.colmenar@urjc.es).

A. J. Nebro is with the Department of Lenguajes y Ciencias de la Computación, ITIS Software, University of Málaga, 29071 Málaga, Spain (e-mail: ajnebro@uma.es).

J. Del Ser is with TECNALIA, Basque Research & Technology Alliance (BRTA), Derio, 48160, Spain, and also with the University of the Basque Country (UPV/EHU), Leioa, 48940, Spain (e-mail: javier.delser@tecnalia.com).

Corresponding author: Alberto Herrán (alberto.herran@urjc.es).

TABLE I
COMPARISON OF PYTHON MOP FRAMEWORKS. ✓ FULL SUPPORT ❍ PARTIAL/LIMITED SUPPORT ✗ ABSENCE

| Feature | pymoo | jMetalPy | DEAP | Platypus | PyGMO | VAMOS |
|---|---|---|---|---|---|---|
| Performance indicators | ✓ | ✓ | ❍ | ✓ | ❍ | ✓ |
| Dense array model | ❍ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Pluggable backends | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Parallelism | ✓ | ✓ | ✓ | ✓ | ✓ | ❍ |
| Dominance-based | ✓ | ✓ | ❍ | ✓ | ✓ | ✓ |
| Decomposition-based | ✓ | ✓ | ✗ | ✓ | ✓ | ✓ |
| Indicator-based | ❍ | ✓ | ✗ | ✓ | ✗ | ✓ |
| Interchangeable operators | ✓ | ✓ | ✓ | ❍ | ✗ | ✓ |
| Conf. steady-state mode | ✓ | ✓ | ❍ | ✗ | ✗ | ✓ |
| External archive | ✗ | ❍ | ❍ | ✓ | ✗ | ✓ |
| Graphical user interface | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |

*PyGMO*. Table I summarizes the main differences between the compared frameworks in terms of architectural features, algorithmic capabilities, configurability, and user-facing tooling. Among the compared frameworks, VAMOS is the only one in this table that combines a dense array population model, pluggable computation backends, and a graphical user interface. *PyGMO* is included for capability context, but in the performance section we treat it separately as a compiled C++ baseline rather than as a directly comparable pure-Python framework.

*pymoo* [6] is a comprehensive and widely used framework offering diverse MOEAs and utilities. While its object-oriented architecture via subclassing promotes customization, it tightly couples algorithmic logic to internal data structures, complicating the systematic vectorization of computations.

*jMetalPy* [7] adapts the Java *jMetal* [4] framework. It provides robust component-based extensibility and runtime monitoring. However, its reliance on collections of individual solution objects prioritizes design clarity over performance, precluding backend-level array vectorization.

*DEAP* [9] provides a generic, highly flexible functional toolkit for prototyping evolutionary algorithms. While powerful, compiling sophisticated multi-objective pipelines requires substantial user effort. Furthermore, its per-individual object model shares the vectorization limits of *jMetalPy*.

*Platypus* [8] is a lightweight, beginner-friendly MOEA library. However, it has seen reduced maintenance activity in recent years, and it relies on object-centric representations similar to *DEAP* and *jMetalPy*.

*PyGMO* [11] wraps the C++ *pagmo2* framework, prioritizing performance and parallel optimization (island model) via compiled extensions. However, its architecture favors continuous single-objective tasks. Its multi-objective capabilities restrict component-level granular tuning and tightly bundle archiving strategies, limiting its utility for algorithm design. We therefore use it later as a compiled baseline for runtime context rather than as a directly comparable pure-Python reference.

Distinct from these central processing unit (CPU)-based Python libraries, recent specialized solutions target divergent architectures and problem scopes. For example, *EvoX* [12] provides a functional programming model specifically tailored for mapping evolutionary workloads onto distributed graphics processing unit (GPU) resources. Other contributions focus on specialized settings, such as coevolutionary constrained optimization [13], bi-knowledge transfer for multimodal problems [14], mixed-integer linear pipelines [15], and large language model (LLM)-assisted code generation [16].

Overall, while existing libraries span a wide spectrum between usability and performance, none explicitly decouple algorithmic logic from the numerical substrate to enable transparent CPU backend substitution. Furthermore, they either target specific niche scenarios or fail to combine a low entry barrier for domain experts with fine-grained configurability for researchers. This gap motivates VAMOS, which provides a vectorized, backend-agnostic architecture aimed specifically at MOEA studies and reproducible cross-framework benchmarking on standard hardware.

## III. VAMOS FRAMEWORK

The design of VAMOS is motivated by three practical gaps identified in existing Python MOEA libraries: *performance*, *configurability*, and *accessibility*.

*a) Performance:* As the population state remains in dense arrays throughout the optimization loop, hot-path computations are dispatched to vectorized or compiled kernels. As we will see in Section V, this design yields substantial runtime reductions across multiple MOEA variants and benchmark families without degrading solution quality. Moreover, switching the compute backend only requires a single-parameter change transparent to algorithm code. Among the compared Python frameworks, no other framework allows swapping the entire computational substrate, from array operations to compiled kernels, without modifying the algorithm implementation.

*b) Configurability:* Most frameworks provide a small set of high-level parameters (population size, crossover/mutation probabilities), but do not expose fine-grained control over offspring batch size, archive attachment, or different initialization methods. VAMOS exposes all of these as first-class configuration options through a builder API. In addition, VAMOS provides a unified tuning interface supporting random search and model-based backends such as Optuna [17], SMAC3 [18], and BOHB [19].

*c) Accessibility:* Most frameworks require importing separate modules for algorithm selection, problem definition, operators, and termination, then wiring them together. This level of boilerplate assumes familiarity with MOEA terminology and framework implementation details, which is a barrier for domain experts who need to solve MOPs but lack a background in metaheuristics. VAMOS addresses this gap at multiple levels (see Section VII): 1) two lines of code are sufficient to run a complete optimization with sensible defaults; 2) a problem factory turns any Python callable into a problem object without subclassing; 3) guided command-line tools and VAMOS Studio provide a clear onboarding path;

and 4) when finer control is needed, the builder API ensures that the framework scales from novice to expert use without a different API.

## IV. VAMOS Architecture

The core architecture of VAMOS is organized into four layers, as shown in Fig. 1: a) the *Foundation Layer* providing compute kernels, problem definitions, quality indicators, encodings, constraints, and evaluation backends; b) the *Engine Layer* implementing MOEAs on top of shared components, variation operators, external archives, and a tuning subsystem; c) the *Experiment Layer* exposing the unified `optimize()` entry point, command-line interface (CLI) tools, benchmarking utilities, and results analysis; and d) the *User Experience (UX) Layer* providing VAMOS Studio (an interactive dashboard), a visual Problem Builder, a Results Explorer, and an onboarding wizard.

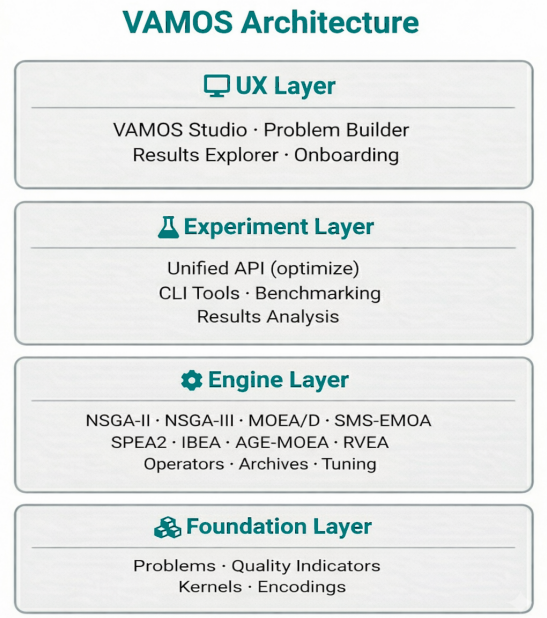


Fig. 1. VAMOS four-layer architecture.

### A. Foundation Layer

Given a MOP with $n$ decision variables and $m$ objectives, the core design choice in VAMOS is to represent a population of size $N$ as a pair of dense arrays, one for decision variables $X \in \mathbb{R}^{N\times n}$ and another for objective values $F \in \mathbb{R}^{N\times m}$. In this way, algorithmic logic is separated from numerical kernels that implement the corresponding core computations. Specifically, VAMOS provides three interchangeable backends to avoid operating over per-individual Python objects. Each backend can be selected through a single configuration parameter, transparently to algorithm code:

1) *NumPy* as a baseline backend enabling vectorized array operations [20].
2) *Numba* as a just-in-time (JIT) compilation of hot operators and utilities, reducing Python overhead [21].
3) *MooCore* as an optional C-backed backend for Pareto ranking and hypervolume-based utilities [22].

In addition to accelerating arithmetic kernels, VAMOS reduces overhead by minimizing Python-side allocations, which allows variation operators to reuse pre-allocated workspaces and algorithms to maintain state as contiguous arrays. This design is particularly beneficial in inner-loop routines such as tournament selection, non-dominated sorting, crowding-distance computation, and archive updates.

Fig. 2 provides an intuitive view of how these frameworks represent and manipulate a population. All frameworks implement the same selection/variation/survival loop, but differ in internal data model and hot-loop granularity. In VAMOS (left) the population is stored in dense arrays ($X$, $F$) and the selection/variation/survival operations are applied via array-oriented kernels (NumPy/Numba/MooCore). In contrast, *pymoo* (center) and *jMetalPy* (right) typically manage the population as a collection of solution objects. Specifically, *pymoo* uses a population container of individual objects backed by NumPy arrays, mixing array computation with object overhead, and *jMetalPy* manages a list of solution objects with per-solution method calls.

Beyond kernels, the Foundation Layer hosts the problem abstraction, a base `Problem` class that declares the number of decision variables, objectives, and optional constraints. A problem registry enables lookup by name (e.g., `"zdt1"`, `"dtlz2"`), and a `make_problem` factory converts any Python callable into a problem object without subclassing. Built-in benchmark families include ZDT [23], DTLZ [24], WFG [25], ZCAT [26], the Congress on Evolutionary Computation (CEC) 2009 test suite [27], and the real-world problem collections RE (Real-World) [28] and RWA (Real-World Applications) [29]. The layer also provides five variable encodings (real, binary, integer, permutation, and mixed), a constraint-handling module with a declarative domain-specific language (DSL), quality indicators (hypervolume, generational distance, inverted generational distance, and spread).

### B. Engine Layer

VAMOS implements the eight MOEAs shown in Table II, covering the three major paradigms: dominance, decomposition, and indicator-based. All eight algorithms are built on top of the shared selection/variation/survival components ensuring consistent operator semantics and enabling component-level substitution without algorithm-specific code changes, so that practitioners can compare paradigms under identical infrastructure and configuration conventions. In addition to the algorithm's selection paradigm, Table II summarizes the supported encodings, number of available crossover and mutation operators, algorithm-specific parameters, and the total number of configurable parameters (base + conditional) in the tuning configuration space. The parameter counts aggregate operators across all supported encodings; in any single configuration with a fixed encoding, between 10 and 18 parameters are simultaneously active. The parameter counts reflect the tuning configuration spaces used by the built-in autotuner (up to 36 for NSGA-II), illustrating the breadth of component-level con-

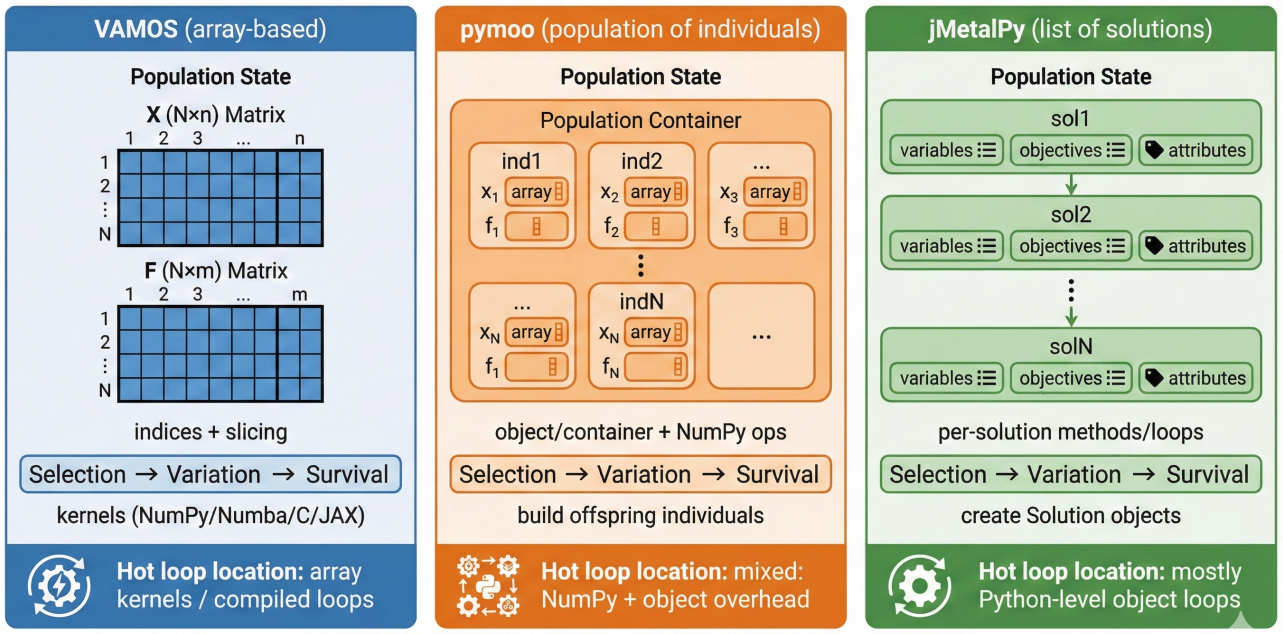


Fig. 2. Comparing population representation across Python MOEA frameworks.

TABLE II
OVERVIEW OF THE EIGHT MOEAS IMPLEMENTED IN VAMOS AND THEIR CONFIGURABLE PARAMETER SPACES ACROSS ALL SUPPORTED ENCODINGS. R = REAL, P = PERMUTATION, B = BINARY, I = INTEGER, M = MIXED

| Algorithm | Ref. | Paradigm | Encodings | #Cx | #Mx | Specific Parameters | #Params |
|---|---|---|---|---|---|---|---|
| NSGA-II | [30] | Dominance | R, P, B, I, M | 25 | 24 | Offspring ratio, repair, external archive (capacity factor, pruning) | 35 |
| NSGA-III | [31] | Dominance | R, P, B, I, M | 21 | 22 | Reference directions (auto-computed) | 22 |
| SPEA2 | [32] | Dominance | R, P, B, I, M | 21 | 22 | Archive size, $k$-nearest neighbors | 24 |
| AGE-MOEA | [33] | Dominance | R, P, B, I, M | 21 | 22 | Archive policies (crowding, HV, $\varepsilon$-grid, hybrid) | 26 |
| MOEA/D | [34] | Decomposition | R, P, B, I, M | 17 | 20 | Aggregation (Tchebycheff, WS, PBI), $T$, $\delta$, $\eta$ | 18 |
| RVEA | [35] | Decomposition | R, P, B, I, M | 21 | 22 | Partitions, $\alpha$ decay, adaptation frequency | 28 |
| SMS-EMOA | [36] | Indicator | R, P, B, I, M | 21 | 22 | Reference point (adaptive) | 22 |
| IBEA | [37] | Indicator | R, P, B, I, M | 21 | 22 | Indicator type, scaling factor $\kappa$ | 24 |

figurability that VAMOS exposes as first-class configuration options.

Algorithms in VAMOS expose a uniform `run(problem, termination, seed, eval_backend, live_viz)` interface, which the high-level `optimize()` entry point delegates to after resolving defaults and configuring operators. All algorithms also support an *ask–tell* loop for streaming or interactive use.

VAMOS supports real-valued, binary, integer, permutation, and mixed-variable encodings through a variation pipeline that composes selection, crossover, mutation, and optional repair operators. For continuous domains, the default configuration uses Simulated Binary Crossover (SBX) and Polynomial Mutation (PM), with mutation probability typically set to $1/n$ where $n$ is the number of decision variables. Binary and integer encodings use standard crossover (one-point, two-point, or uniform) and bit-flip or bounded mutation, while permutation encodings provide order and cycle crossover with insertion mutation. Mixed-variable problems combine encoding-specific operators transparently. Encodings are normalized internally to ensure consistent operator resolution and avoid silent configuration mismatches.

The engine layer also provides optional external archives, both bounded (with crowding-distance or hypervolume-based pruning) and unbounded, that accumulate non-dominated solutions independently of the evolving population. This decouples the population-based search from the elite set, which is particularly useful for problems with more than two objectives [10]. Common quality indicators are provided by the foundation layer, including hypervolume, generational distance (GD), inverted generational distance (IGD), and spread. Hypervolume computation can leverage the MooCore C backend when available (MooCore is an optional dependency because it requires C compilation, which may not be available on all platforms), with pure-Python fallback implementations for low-dimensional cases.

### C. Experiment Layer

The experiment layer hosts the unified `optimize()` entry point that resolves defaults, instantiates the algorithm and problem, and delegates to the engine. It also provides a command-line interface with subcommands for running

optimizations, hyperparameter tuning, ablation studies, diagnostics, and launching VAMOS Studio. A benchmark runner executes standardized multi-seed studies under a shared protocol (fixed problem definitions, evaluation budgets, and seeds) and emits machine-readable artifacts (per-seed objective values, summary CSVs, and LaTeX-ready tables), reducing transcription errors and enabling end-to-end reproducibility.

### D. User Experience Layer

The user experience layer provides *VAMOS Studio*, a Panel-based [38] dashboard designed to make multi-objective optimization accessible to users with limited programming experience. It offers three main views: (i) a *Welcome* tab with a first-launch onboarding wizard and quick-reference tables for the CLI and Python API; (ii) a *Problem Builder* that lets users write objective and constraint functions in the browser, adjust bounds and algorithm settings, and see the Pareto front update live after each preview run; and (iii) an *Explore Results* tab for loading completed studies, comparing algorithms, and ranking solutions with multi-criteria decision-making methods. The *Problem Builder* ships with domain-specific templates (engineering design, machine learning, and scheduling) so that domain experts can start from a realistic example rather than a blank editor. Generated problems can be exported as standalone Python scripts or run directly through the API.

In addition to Studio, the UX layer includes analysis utilities for statistical comparison of algorithm runs (Wilcoxon tests, effect sizes), visualization helpers for Pareto fronts and convergence plots, and multi-criteria decision-making scorers (TOPSIS, weighted sums).

## V. Performance

This section describes the experimental setup used to evaluate VAMOS, which is centered on assessing its computational performance compared with other frameworks. It is worth mentioning that we will focus on continuous optimization problems, as analyzing other encodings is out of the scope of this paper because of space constraints. We define the benchmark suite, justify the choice of compared frameworks and algorithms, describe the cross-framework semantic-alignment protocol, and specify the runtime measurement and quality-indicator protocols.

### A. Compared Frameworks and Algorithms

Although VAMOS implements eight MOEAs, our cross-framework comparison focuses on those with the closest semantic matches across libraries: NSGA-II, SMS-EMOA, and MOEA/D. This scope avoids confounding effects from missing implementations or materially different operator and termination semantics in specific frameworks. Furthermore, these algorithms are the most representative of the three commonly accepted categories of MOEAs: dominance-based (NSGA-II), indicator-based (SMS-EMOA), and decomposition-based (MOEA/D).

In the case of NSGA-II, we compare three variants: the baseline (i.e., standard generational NSGA-II), a steady-state version, and an unbounded archive-based version. NSGA-II is a generational genetic algorithm, but some studies [39] have shown that a steady-state version (i.e., the offspring population contains only one solution) produces fronts with improved diversity compared to the standard version. NSGA-II has traditionally been considered a poorly performing algorithm for problems with more than two objectives, but it was reported in [10] that incorporating an unbounded external archive improves the performance of NSGA-II on three-objective problems. Both variants enhance the base NSGA-II but at the cost of increased computational overhead.

We include DEAP, Platypus, and PyGMO only in an experiment that involves the standard generational NSGA-II. For the remaining experiments, our choice of algorithms restricts the comparison of VAMOS to the two previously analyzed pure-Python multi-objective frameworks with the closest semantic matches, namely pymoo and jMetalPy. DEAP does not include built-in implementations of SMS-EMOA or MOEA/D, Platypus lacks SMS-EMOA and has seen reduced maintenance activity in recent years, and PyGMO does not include MOEA/D and is restrictive in terms of algorithmic configurability.

### B. Benchmark Suites

We evaluate runtime and solution quality on widely used synthetic benchmarks: ZDT1–4 and ZDT6 [23] (two objectives; ZDT5 is excluded because it uses a binary encoding), DTLZ1–7 [24] (three objectives), WFG1–9 [25] (two objectives) and the recent ZCAT test suite [26] (two objectives). Unless otherwise stated, each (problem, framework) configuration is executed for 50,000 objective evaluations with 30 independent seeds. We have fixed the number of evaluations to simplify the comparisons, although it is well-known that the stopping condition to solve the ZDT and bi-objective WFG benchmarks is typically 20,000-25,000 evaluations.

### C. Cross-Framework Semantic Alignment

Fig. 3 summarizes the benchmark protocol and semantic-alignment workflow used to ensure cross-framework fairness and to support the "same quality, faster" claim for the aligned NSGA-II comparison.

To ensure comparability across frameworks, we standardize the algorithmic settings of NSGA-II: population size $N = 100$, SBX crossover probability $p_c = 1.0$ and distribution index $\eta_c = 20$, polynomial mutation distribution index $\eta_m = 20$ and mutation probability $p_m = 1/n$, and binary tournament selection. For each framework we map these settings to the closest available implementation. The initial population is filled with randomly initialized solutions.

For SMS-EMOA, we use the same common settings as NSGA-II. Compared with this algorithm, its main differences are the use of hypervolume contribution as density estimator, random parent selection, and a steady-state $(\mu + 1)$ loop (one offspring per iteration). However, the hypervolume-contribution reference point is not defined consistently across frameworks: VAMOS and jMetalPy compute contributions in raw objective space with an adaptive reference point $r =$

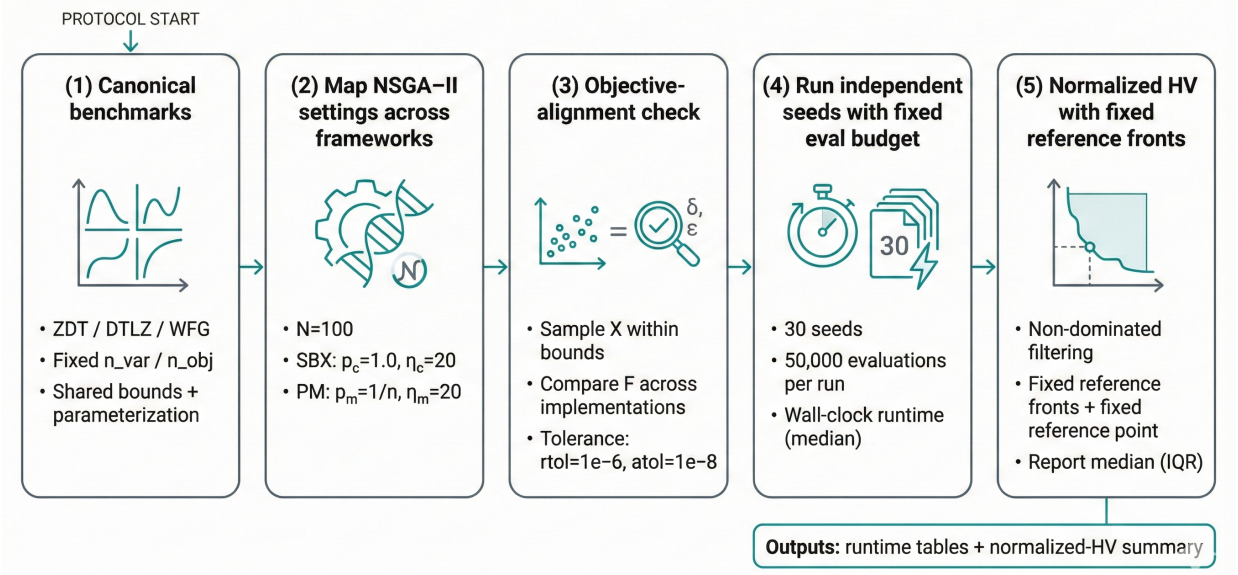


Fig. 3. Benchmark protocol and semantic alignment: map equivalent NSGA-II settings across frameworks, run independent seeds under a fixed evaluation budget, and compute normalized hypervolume using fixed reference fronts.

$\max(F) + 1$, while pymoo normalizes objectives and uses a fixed shifted reference point $r = \mathbf{1} + \epsilon$ (we set $\epsilon = 1$). We therefore report the cross-framework SMS-EMOA comparison with this caveat.

For MOEA/D, we align population size $N = 100$, weight vectors, neighborhood size $T = 20$, neighborhood selection probability $\delta = 0.9$, and replacement limit $\eta = 20$ . We use penalty-based boundary intersection (PBI) scalarization with $\theta = 5$ and the SBX and polynomial mutation operators with the same parameters as NSGA-II. The weight vectors for the three-objective problems are generated using the Riesz $s$-energy method proposed in [40], available at https://www.egr.msu.edu/coinlab/blankjul/uniform/.

This workflow follows standard practice in comparative MOEA studies (independent runs, quality indicator computation, and structured reporting) and is consistent with the experimentation methodology described in jMetalPy [7]. Our main extension is to make the *inputs* to this workflow comparable across frameworks by enforcing semantic alignment of both problem definitions and operator probabilities.

Crucially, we align *semantics* (not only parameter names) and enforce consistent benchmark definitions. For example, in pymoo the polynomial mutation operator exposes both an individual-level probability (`prob`) and a per-variable probability (`prob_var`); to match the standard "$p_m = 1/n$ per decision variable" setting used by jMetalPy, we set `prob`=1 and `prob_var`=1/n.

For WFG we adopt pymoo's canonical parameterization for two objectives ($k = 4$, $l = 20$ for $n = 24$) and pass parameters explicitly when a toolkit exposes different defaults. While full equivalence cannot be guaranteed due to framework-specific details (tie-breaking, boundary handling, duplicate elimination, etc.), these choices aim to make the comparison as fair as possible by removing confounders unrelated to algorithmic overhead and numerical kernels.

### D. Runtime Measurement

All runtimes are measured wall-clock using high-resolution timers and include algorithm overhead and objective evaluations.

For Numba-accelerated backends, we distinguish two timing policies. **Warm** timings exclude one-time JIT compilation by performing a short warmup run in the same process before starting the timer (2,000 warmup evaluations; not counted in the reported time). **Cold** timings measure the first run from a fresh process and therefore include JIT compilation overhead. Unless otherwise stated, we report warm timings to reflect steady-state throughput; cold-start costs can be reproduced with the provided scripts.

Experiments were run on an MSI Stealth 16 AI Studio A1VGG laptop with an Intel(R) Core(TM) Ultra 9 185H central processing unit (CPU) (16 cores, 22 logical processors) and 32 GB random-access memory (RAM), running Microsoft Windows 10 Home (version 25H2, build 26200). We used Python 3.12.3 and evaluated VAMOS 0.1.0 (NumPy 2.3.5 with OpenBLAS 0.3.30; Numba 0.63.1) against pymoo 0.6.1.6 and jMetalPy 1.9.0.

### E. Quality Indicators

Hypervolume (HV), denoted $\mathrm{HV}(A, r)$, measures the Lebesgue measure of the region dominated by an approximation set $A$ with respect to a reference point $r$ (for minimization) [41]. Because HV is scale-dependent and sensitive to the choice of $r$, naive implementations can produce values that are not comparable across runs (e.g., if $r$ is expanded per run) and can be misleading when dominated solutions are included. To ensure a stable and comparable quality metric, we adopt the following protocol:

1) **Non-dominated filtering**: for every framework we compute HV on the final non-dominated set only.

2) **Fixed reference Pareto fronts**: for each benchmark problem we store a dense reference Pareto front $P_{\text{ref}}$. For ZDT and DTLZ (except DTLZ7) these fronts are generated analytically; for DTLZ7 and WFG we generate $P_{\text{ref}}$ by dense sampling followed by non-dominated filtering.
3) **Fixed reference point**: we set $r = \max(P_{\text{ref}}) + \epsilon$ (component-wise) with a small $\epsilon$, and we disallow run-dependent expansion.
4) **Normalization**: we report $\text{HV}(A, r)/\text{HV}(P_{\text{ref}}, r)$, yielding a dimensionless score typically in $[0, 1]$ and reducing sensitivity to objective scaling.

For WFG problems, the reference front is necessarily an approximation; we use large Pareto-set samples and non-dominated filtering, and we increase density for particularly sensitive cases (e.g., WFG2) to avoid underestimating $\text{HV}(P_{\text{ref}}, r)$ and obtaining normalized HV slightly above 1.

### F. VAMOS Backend Comparison

In the first experiment, we compare the performance of NSGA-II using the different backends of VAMOS on the ZDT, DTLZ, WFG, and ZCAT families. For each family, we sum the execution times of solving all the problems in the family and report the median runtime of repeating this process 30 times. The results are included in Table III.

TABLE III
VAMOS BACKEND COMPARISON: MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY

| Backend | ZDT | DTLZ | WFG | ZCAT | Average |
|---|---|---|---|---|---|
| Numba | **4.89** | **8.14** | **14.33** | **87.43** | **28.70** |
| MooCore | 21.01 | 36.45 | 53.96 | 127.46 | 59.72 |
| NumPy | 35.14 | 68.86 | 52.68 | 177.06 | 83.44 |

The Numba backend is the fastest across all four families. MooCore is 1.5–4.5× slower than Numba, while the pure NumPy backend is 2.0–8.5× slower, confirming that per-generation interpreted Python overhead dominates the cost in the absence of JIT compilation. The largest gap appears on DTLZ, where NumPy is 8.5× slower than Numba; the smallest relative difference is between Numba and MooCore on ZCAT (1.5×).

We proceed next to quantify how the array-native architecture scales when the population size increases, so we measure wall-clock runtimes of the Numba and NumPy backends on three representative problems (ZDT4, DTLZ2, WFG2) using NSGA-II with 50,000 evaluations and 5 independent seeds per configuration. Population sizes range from 50 to 800; all other settings match the main benchmark protocol.

Fig. 4 shows that the runtime gap between NumPy and Numba widens substantially as the population grows. At $N = 50$ the two backends perform similarly (and NumPy is even marginally faster on ZDT4, likely because the JIT warm-up cost is not fully amortized for small arrays), but by $N = 800$ the NumPy backend requires about 30 s on all three problems while Numba stays below 12 s—a roughly 2.5× speedup. The effect is consistent across problem families despite their different dimensionalities ($n_{\text{var}} = 10$, 12, and 24 for ZDT4, DTLZ2, and WFG2 respectively). Crucially, the Numba curve remains nearly flat across population sizes, confirming that the JIT-compiled kernels keep per-generation overhead nearly constant regardless of population size, whereas NumPy's interpreted dispatch overhead accumulates with each generation cycle. These results reinforce the practical relevance of the vectorized design for large-scale benchmarking studies that require large populations or many independent seeds under a fixed compute budget.

### G. Cross-Framework Comparison

In this section, we first compare the runtime of VAMOS against other Python frameworks using generational NSGA-II. Then, we extend the comparison to NSGA-II variants (steady-state and external archive), SMS-EMOA, and MOEA/D.

Table IV compares the runtime of generational NSGA-II across five pure-Python frameworks and PyGMO, a compiled C++ baseline (Python bindings to the PaGMO2 library). Among pure-Python frameworks, VAMOS (Numba backend) is the fastest in all four families (ZDT, DTLZ, WFG, and ZCAT). jMetalPy is the closest competitor on ZDT, DTLZ, and ZCAT, yet it remains 2.2–4.9× slower on those families and 29.4× slower on WFG. pymoo is 2.7–8.2× slower, while DEAP and Platypus are 4.3–82.9× slower. PyGMO remains faster than VAMOS on ZDT, DTLZ, and WFG (1.3–1.4×), as expected given that both its algorithms and benchmark functions execute entirely in compiled C++ code; however, VAMOS is slightly faster on ZCAT (87.43 s vs. 98.55 s).

TABLE IV
NSGA-II WITH STANDARD SETTINGS. MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY ACROSS ALL FRAMEWORKS; PYGMO (C++) IS INCLUDED AS A COMPILED BASELINE

| Framework | ZDT | DTLZ | WFG | ZCAT |
|---|---|---|---|---|
| VAMOS | **4.89** | **8.14** | **14.33** | **87.43** |
| pymoo | 39.90 | 60.43 | 77.12 | 231.29 |
| jMetalPy | 24.09 | 32.30 | 421.20 | 196.02 |
| DEAP | 163.78 | 226.99 | 1154.80 | 378.59 |
| Platypus | 203.58 | 270.24 | 1188.44 | 445.80 |
| PyGMO (C++) | **3.41** | **5.69** | **11.07** | **98.55** |

We now compare the runtime of NSGA-II variants (steady-state and external unbounded archive), SMS-EMOA, and MOEA/D across the three frameworks that support them. The median runtimes are reported in Table V. In the *steady-state* variant of NSGA-II, only one offspring is produced per iteration, so the main loop executes $N$ times more iterations than the generational variant for the same evaluation budget, introducing additional per-iteration overhead. The variant with an *external unbounded archive* implies that whenever a new solution is evaluated, it is compared with the archive: if the new solution is non-dominated, it is inserted and any archive members it dominates are removed. This process becomes more time-consuming as the archive grows. Once the algorithm terminates, a subset of $N$ diverse solutions is selected from the archive and returned as the final approximation set.

SMS-EMOA uses the hypervolume contribution as density estimator to select the worst individual to remove from the

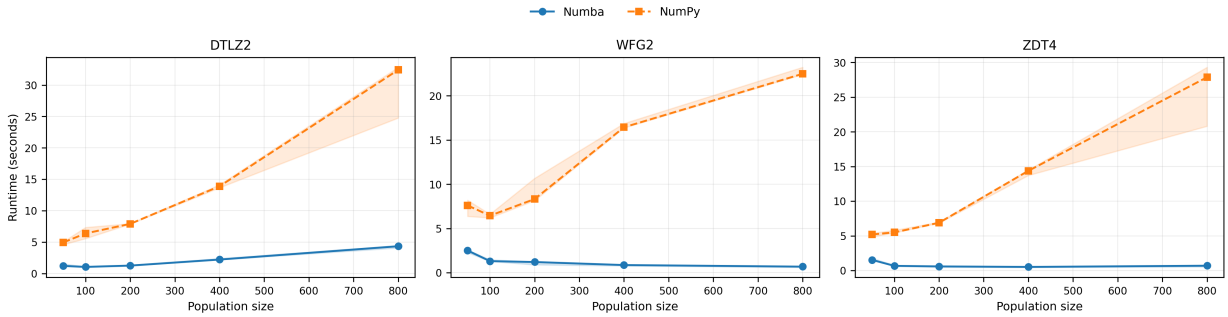


Fig. 4. Runtime scaling of VAMOS backends (Numba vs. NumPy) as a function of population size on three representative problems. Shaded regions indicate the interquartile range across 5 seeds. NumPy runtime grows steeply with population size due to increasing Python-level overhead in per-generation operations, while Numba runtime remains nearly flat, demonstrating that JIT-compiled kernels amortize loop overhead effectively on larger arrays.

population in the replacement step, which is a time-consuming procedure given that the computation of the hypervolume contribution grows exponentially with the number of objectives. MOEA/D uses a decomposition strategy that, a priori, should not introduce a noticeable overhead.

TABLE V
MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY FOR NSGA-II VARIANTS, SMS-EMOA, AND MOEA/D.

| Algorithm | Framework | ZDT | DTLZ | WFG | ZCAT |
|---|---|---|---|---|---|
| NSGA-II (steady-state) | VAMOS | **150.21** | **244.80** | **799.29** | **715.29** |
| | pymoo | 5747.47 | 7259.88 | 6382.73 | 4569.01 |
| | jMetalPy | 1930.43 | 2506.83 | 2756.07 | 1947.76 |
| NSGA-II (ext. archive) | VAMOS | **10.43** | **87.56** | **23.33** | **106.30** |
| | pymoo | 1422.71 | 7208.12 | 1515.89 | 1511.36 |
| | jMetalPy | 744.93 | 3993.47 | 1736.74 | 602.88 |
| SMS-EMOA | VAMOS | **124.94** | **187.16** | **734.42** | **521.22** |
| | pymoo | 1658.10 | 2439.60 | 3410.49 | 2967.14 |
| | jMetalPy | 377.01 | 529.49 | 1161.51 | 2670.22 |
| MOEA/D | VAMOS | **76.19** | **118.98** | **647.96** | **340.78** |
| | pymoo | 1325.29 | 1735.46 | 4143.63 | 1023.63 |
| | jMetalPy | 303.00 | 430.14 | 4267.52 | 354.29 |

VAMOS is the fastest framework for all four algorithm variants across all problem families. The speedup is most pronounced for the steady-state variant, where VAMOS is 6.4–38.3× faster than pymoo and 2.7–12.9× faster than jMetalPy. These large margins are expected: steady-state algorithms execute $N$ iterations per generation, amplifying the per-iteration overhead of object-oriented frameworks. For the external-archive variant, VAMOS is 14.2–136.4× faster than pymoo and 5.7–74.4× faster than jMetalPy. For MOEA/D, VAMOS is 3.0–17.4× faster than pymoo and 1.0–6.6× faster than jMetalPy, with the narrowest gap on ZCAT. For SMS-EMOA, VAMOS is 4.6–13.3× faster than pymoo and 1.6–5.1× faster than jMetalPy, with the smallest margin on WFG.

Peak resident-set-size (RSS) measurements for NSGA-II on representative problems are reported in the supplementary material. They are consistent with the array-native representation reducing memory overhead relative to object-centric frameworks.

### H. Convergence, Solution Quality, and Statistical Analysis

To verify that the runtime speedups do not compromise solution quality, the supplementary material provides three complementary analyses. Appendix A presents a convergence analysis: normalized hypervolume is tracked at 1,000-evaluation intervals over 50,000 evaluations on three representative problems (DTLZ2, WFG4, ZDT1), showing that VAMOS, pymoo, and jMetalPy follow nearly indistinguishable trajectories. Appendix B reports the solution quality comparison: all frameworks attain virtually identical normalized HV on ZDT, medians are tightly clustered on DTLZ, and VAMOS achieves the highest family-level median on WFG for the NSGA-II baseline (0.934 vs. 0.846 for pymoo in the supplementary summary). Appendix C details the statistical analysis: for the 41 NSGA-II benchmark problems, Wilcoxon signed-rank tests with Holm correction reveal only one significant difference (ZDT6, $\Delta \approx 0.4\%$), and paired-bootstrap equivalence tests confirm that VAMOS is equivalent to pymoo on 37/41 problems and non-inferior on 38/41. Taken together, these results confirm that VAMOS delivers faster runtimes without sacrificing convergence behavior or final solution quality.

### I. Discussion

The observed speedups are primarily due to (i) the array-based representation that removes per-solution Python overhead, and (ii) JIT compilation of hot operators and utilities in the Numba backend. The scalability study (Fig. 4) confirms this mechanism: the NumPy backend's runtime grows steeply with population size, reflecting per-generation interpreted overhead, while the Numba backend remains nearly flat. The solution quality analysis in Appendix B of the supplementary material summarizes that these performance gains are not obtained at the expense of final solution quality under the fixed-reference-front normalized-HV protocol, avoiding artifacts caused by run-dependent reference points.

The cross-framework gaps are problem-family dependent: WFG problems exhibit the largest relative slowdowns for object-centric libraries, consistent with their higher dimensionality amplifying per-solution overhead. The magnitude of the speedup also varies across algorithm variants in a pattern consistent with this overhead model. Steady-state variants

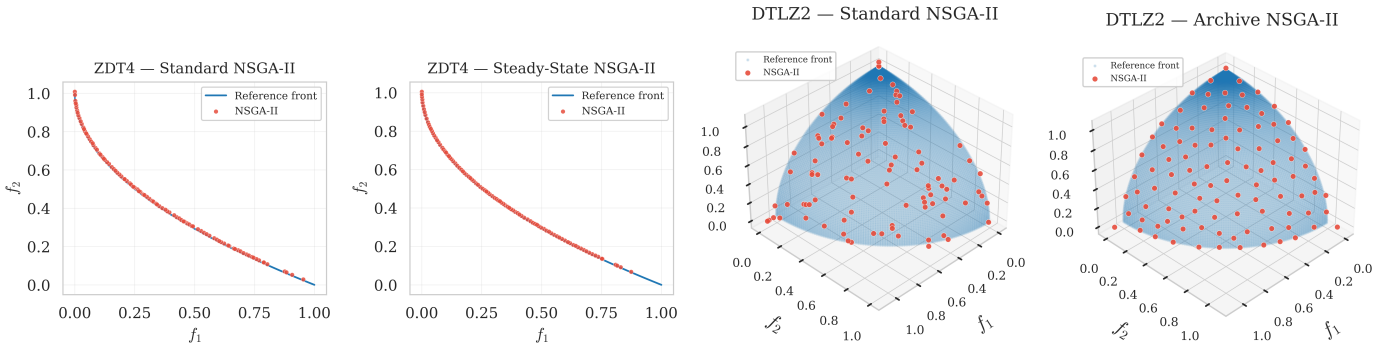


Fig. 5. Final non-dominated sets produced by NSGA-II variants on ZDT4 (left pair, 2-objective) and DTLZ2 (right pair, 3-objective), overlaid with the reference Pareto front. From left to right: (a) standard generational NSGA-II on ZDT4, (b) steady-state NSGA-II on ZDT4, (c) standard NSGA-II on DTLZ2, and (d) NSGA-II with unbounded external archive on DTLZ2. The steady-state and archive variants produce a more uniformly distributed approximation of the Pareto front compared to the generational baseline.

(Table V) show the largest relative speedups because they perform one selection–variation–survival cycle per offspring: frameworks with high per-iteration Python overhead incur that cost 50,000 times (one per evaluation) rather than 500 times (one per generation of 100 offspring). VAMOS's array-kernel dispatch amortizes this overhead more effectively, widening the gap.

Not all comparisons yield equally large margins. The smallest gap appears for SMS-EMOA on WFG, where jMetalPy is only 1.6× slower than VAMOS, and for MOEA/D on ZCAT, where the gap narrows to near parity (1.04×). Unlike steady-state NSGA-II—where VAMOS's Numba-accelerated ranking and crowding-distance computation dominate the per-iteration cost—SMS-EMOA's survival selection is bottlenecked by the hypervolume contribution calculation on the worst-ranked front. Both frameworks delegate this computation to the same C library (`moocore`), so VAMOS's kernel acceleration yields a smaller differential advantage in that setting. Similarly, MOEA/D spends most of its time in neighborhood replacement rather than in generic ranking kernels, which limits the benefit of the backend distinction on ZCAT.

These multi-fold runtime reductions have direct practical implications. They mean that the same compute budget can accommodate proportionally more independent seeds, larger population sizes, or denser hyperparameter sweeps—precisely the conditions required for statistically rigorous benchmarking studies. Moreover, the scaling study (Fig. 4) shows that the advantage grows with population size, so the benefit is amplified in exactly the large-scale regimes where it matters most.

### J. Threats to Validity

Despite careful alignment of dimensions, budgets, problem definitions, and operator settings, cross-framework comparisons remain subject to several threats: (i) operator implementations may differ in subtle details (e.g., boundary handling, duplicate elimination, tie-breaking in survival), (ii) random number generation and seeding semantics vary across libraries, and (iii) library-specific overheads (data conversion, object materialization) may affect measured runtime. We mitigate major sources of unfairness by enforcing consistent benchmark bounds and matching operator semantics where APIs differ (e.g., mutation probability in pymoo).

Our cross-framework comparison focuses on Python MOEA libraries whose inner loops are predominantly executed in Python. PyGMO (Python bindings to the C++ PaGMO2 library) is included as a compiled baseline for reference (Table IV), but it executes algorithms and benchmark functions fully in compiled code, yielding fundamentally different runtime profiles; its results are therefore not directly comparable with those of the pure-Python frameworks.

For hypervolume, reference fronts for problems without closed-form Pareto fronts are necessarily approximations; while we generate them densely and fix them across runs, remaining discrepancies can affect absolute normalized values, especially on WFG problems. We therefore treat normalized HV primarily as a comparative metric under a fixed protocol, and we report distributions across multiple seeds rather than relying on single-run outcomes.

## VI. Configurability

This section examines two complementary aspects of VAMOS's configurability. Section VI-A analyzes the parameter space of NSGA-II across frameworks, and Section VI-B describes how VAMOS integrates automatic algorithm configuration tools to systematically explore that space.

### A. Parameter Space Analysis of NSGA-II

We examine the configurability of VAMOS by analyzing the parameter space of NSGA-II with real-valued encoding. Table VI lists the 36 configurable parameters organized by component (general settings, initialization, selection, crossover, mutation, and repair). Of these, 10 are always active (e.g., population size, repair strategy, crossover type and probability, mutation type and probability factor) and 26 are conditional, i.e., they are activated only when their parent operator is selected (e.g., the distribution index $\eta_c$ applies only when SBX crossover is chosen). In any single configuration, between 10 and 18 parameters are simultaneously active, depending on the selected operators.

Table VII compares this breadth across frameworks (detailed per-framework tables are provided in the supplementary

TABLE VI
SUMMARY OF THE CONSIDERED PARAMETER SPACE FOR NSGA-II (REAL-VALUED ENCODING). PARAMETERS IN ITALICS ARE CONDITIONAL, ACTIVE ONLY WHEN THEIR CORRESPONDING OPERATOR IS SELECTED. VALUES IN **BOLD** INDICATE THE DEFAULT PARAMETERS OF NSGA-II. THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS IS 36. ACRONYMS: LHS (LATIN HYPERCUBE SAMPLING), OBL (OPPOSITION-BASED LEARNING), SBX (SIMULATED BINARY CROSSOVER), BLX (BLEND CROSSOVER), PCX (PARENT CENTRIC CROSSOVER), UNDX (UNIMODAL NORMAL DISTRIBUTION CROSSOVER), SPX (SIMPLEX CROSSOVER), SUS (STOCHASTIC UNIVERSAL SAMPLING). THE STEADY-STATE SETTING CORRESPONDS TO PRODUCING ONE OFFSPRING PER ITERATION.

| Component | Value / Strategy | Conditional Parameters |
|---|---|---|
| General | Pop. Size $\in [20, 200]_{\mathbb{Z}}$ (log-scale); default $\mathbf{100}$ | – |
| | Offspring Update $\in \{\text{steady-state}, 0.25, 0.5, 0.75, \mathbf{1.0}\}$ | – |
| | Use External Archive $\in \{\text{True}, \textbf{False}\}$ | – |
| | *Archive Mode* $\in \{\textbf{bounded}, unbounded\}$ | *Archive* = True |
| | *Archive Prune Policy* $\in \{\textbf{crowding}, \text{hv}, \text{mc\_hv}, \text{knn}, \text{maxmin}, \text{ref\_dirs}\}$ | *Archive* = True, *Mode* = bounded |
| Initialization | $\{\textbf{Random}, \text{LHS}, \text{Scatter}, \text{Sobol}, \text{Halton}, \text{OBL}\}$ | – |
| | *Scatter Base Size Factor* $\in \{0.1, 0.2, 0.3, 0.5, 0.75, 1.0\}$ | *Init.* = Scatter |
| Selection | $\{\textbf{Tournament}, \text{Random}, \text{Boltzmann}, \text{Ranking}, \text{SUS}\}$ | – |
| | *Tournament Size* $\in [\mathbf{2}, 10]_{\mathbb{Z}}$ | *Selection* = Tournament |
| Crossover | *Global:* Probability $\in [0.6, 1.0]_{\mathbb{R}}$ | |
| | **SBX** | *Dist. Index* $\in [5.0, 40.0]_{\mathbb{R}}$ |
| | BLX-$\alpha$ | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$ |
| | BLX-$\alpha\beta$ | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$, $\beta \in [0.0, 1.0]_{\mathbb{R}}$ |
| | Arithmetic | – |
| | Whole Arithmetic | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$ |
| | Laplace | $a \in [-1.0, 1.0]_{\mathbb{R}}$, $b \in [0.01, 2.0]_{\mathbb{R}}$ |
| | Fuzzy | $d \in [0.0, 2.0]_{\mathbb{R}}$ |
| | PCX | $\sigma_\eta \in [0.01, 0.5]_{\mathbb{R}}$, $\sigma_\zeta \in [0.01, 0.5]_{\mathbb{R}}$ |
| | UNDX | $\zeta \in [0.1, 1.0]_{\mathbb{R}}$, $\eta \in [0.1, 1.0]_{\mathbb{R}}$ |
| | Simplex | $\epsilon \in [0.1, 1.0]_{\mathbb{R}}$ |
| Mutation | *Global:* Prob. Factor $\in [0.25, 3.0]_{\mathbb{R}}$ ($p_m = \text{factor}/n$) | |
| | **Polynomial** | *Dist. Index* $\in [5.0, 40.0]_{\mathbb{R}}$ |
| | Linked Polynomial | *Dist. Index* $\in [5.0, 40.0]_{\mathbb{R}}$ |
| | Non-Uniform | *Perturbation* $\in [0.05, 0.5]_{\mathbb{R}}$ |
| | Gaussian | $\sigma \in [0.001, 0.5]_{\mathbb{R}}$ |
| | Cauchy | $\gamma \in [0.001, 0.5]_{\mathbb{R}}$ |
| | Uniform | *Perturbation* $\in [0.01, 0.5]_{\mathbb{R}}$ |
| | Uniform Reset | – |
| | Lévy Flight | $\beta \in [0.5, 2.0]_{\mathbb{R}}$, *Scale* $\in [0.001, 0.1]_{\mathbb{R}}$ |
| | Power Law | *Index* $\in [0.5, 5.0]_{\mathbb{R}}$ |
| Repair | $\{\textbf{clip}, \text{reflect}, \text{random}, \text{round}, \text{wrap}, \text{midpoint}\}$ | After crossover and mutation operations. |

material). VAMOS offers 41 strategies and operators across the six components, roughly $3\times$ the variety of pymoo (14) and $1.6\times$ that of jMetalPy (26), with 26 conditional parameters that capture operator-specific settings. The largest gap appears in crossover (10 operators vs. 3 in pymoo) and mutation (9 vs. 2). Furthermore, neither competing framework exposes all configuration options as first-class parameters through a unified API: pymoo requires a custom `Callback` for external archives, while jMetalPy needs an evaluator wrapper. VAMOS exposes all of them as builder-level options, enabling the systematic configuration-space exploration described in the next section.

The supplementary material contains the full parameter space of all the MOEAs included in VAMOS and an explanation of the parameters of each algorithm.

TABLE VII
NUMBER OF AVAILABLE STRATEGIES AND OPERATORS FOR NSGA-II (REAL-VALUED ENCODING) BY COMPONENT ACROSS FRAMEWORKS. FOR GENERAL, ENTRIES DENOTE THE NUMBER OF CONFIGURABLE PARAMETERS; FOR ALL OTHER COMPONENTS, ENTRIES DENOTE THE NUMBER OF AVAILABLE STRATEGIES OR OPERATORS. THE NUMBER OF CONDITIONAL PARAMETERS IS SHOWN IN PARENTHESES WHERE APPLICABLE.

| Component | VAMOS | pymoo | jMetalPy |
|---|---|---|---|
| General | 5 (2) | 2 | 5 (2) |
| Initialization | 6 (1) | 2 | 2 |
| Selection | 5 (1) | 2 (1) | 3 (1) |
| Crossover | 10 (13) | 3 (6) | 6 (9) |
| Mutation | 9 (9) | 2 (2) | 6 (7) |
| Repair | 6 | 3 | 4 |
| **Total** | **41 (26)** | 14 (9) | 26 (19) |

### *B. Automatic Algorithm Configuration*

The parameter spaces described in the previous section highlight a practical challenge: with up to 36 configurable parameters (spanning 41 strategies and operators) for NSGA-II, manually finding a good configuration for a given problem or problem family is impractical. Even experienced practitioners typically rely on well-known defaults (e.g., SBX with $\eta_c = 20$, polynomial mutation with $p_m = 1/n$), which may be far from optimal for specific problem characteristics. Automatic algorithm configuration addresses this challenge by systematically exploring the configuration space and identifying high-performing settings without manual intervention.

The optimization and machine learning communities have developed several well-established tools for this purpose. Optuna [17] is a widely adopted hyperparameter optimization framework that uses Tree-structured Parzen Estimators (TPE) and other Bayesian methods to efficiently navigate large search spaces. SMAC3 [18] combines Random Forest surrogate models with an aggressive racing mechanism, and has a long track record in the algorithm configuration literature, particularly for high-dimensional spaces with conditional parameters. BOHB [19] merges Bayesian optimization with HyperBand's multi-fidelity scheduling, allocating computational budget efficiently by evaluating many configurations cheaply at low fidelity and progressively investing more resources in the most promising candidates.

Rather than expecting users to install, configure, and interface with these tools independently, VAMOS integrates all three as tuning backends accessible through a unified API. The `ModelBasedTuner` facade provides a single entry point: users select the desired backend with a single parameter (`backend="optuna"`, `"smac3"`, or `"bohb"`) and VAMOS handles the translation between its internal configuration space and each backend's native format. This means that conditional parameter dependencies (such as the SBX distribution index being active only when SBX is the selected crossover operator) are automatically encoded in the backend's search space without requiring any manual definition from the user.

Optuna offers the broadest selection of sampling strategies (TPE, CMA-ES, Gaussian Process, NSGA-II/III, Quasi-Monte Carlo) and supports distributed tuning via database-backed studies. SMAC3 excels in mixed configuration spaces where categorical, integer, and continuous parameters coexist with conditional dependencies. BOHB is particularly well suited when the evaluation budget varies, as its HyperBand scheduling automatically decides how many resources to allocate to each configuration.

## VII. Accessibility

A key design goal of VAMOS is to lower the barrier to entry for multi-objective optimization without sacrificing the granular control required by expert researchers. We operationalize accessibility here through three onboarding tasks aligned with common first-use workflows: executing a built-in benchmark, defining a small custom problem, and moving from defaults to explicit algorithm configuration. For VAMOS, pymoo, and jMetalPy, we curated canonical snippets from the shortest documented path for each task and use them as objective accessibility proxies.

Fig. 6 summarizes this progressive-disclosure workflow. The same framework supports a two-line quick start for built-in problems, callable-based custom problem definition, expert-level configuration through the Builder API, and guided tooling through the command-line interface and VAMOS Studio.

Table VIII summarizes the resulting proxy measurements. VAMOS has the lowest setup cost for the initial built-in run and for the callable-based custom-problem task, while pymoo becomes slightly shorter once the workflow shifts to explicit operator-level configuration. We interpret this pattern as evidence for progressive disclosure rather than universal terseness: VAMOS minimizes the initial setup burden and then scales to expert control without leaving the public API, while also providing guided tooling beyond the Python interface.

```python
from vamos import optimize

result = optimize("zdt1", algorithm="nsgaii")
```

Listing 1. Minimal NSGA-II run on ZDT1 in VAMOS.

```python
from vamos import make_problem, optimize

def my_objectives(x):
    f1 = x[0]**2 + x[1]**2
    f2 = (x[0] - 1)**2 + (x[1] - 1)**2
    return [f1, f2]

problem = make_problem(
    my_objectives, n_var=2, n_obj=2,
    bounds=[(0, 2), (0, 2)],
    encoding="real",
)
result = optimize(
    problem, algorithm="nsgaii",
    max_evaluations=50000, seed=42
)
```

Listing 2. Defining and optimizing a custom problem.

Listing 1 shows the corresponding minimal VAMOS example. Executing a standard algorithm with sensible defaults on a built-in benchmark requires only two lines of code.

For custom problems, `make_problem` converts a standard Python callable into a problem object without introducing framework-specific base classes, as shown in Listing 2.

For advanced users who need systematic algorithm configuration and experimentation, VAMOS provides a unified Builder API. Listing 3 illustrates how a researcher can define the population size, configure specific crossover and mutation operators, and attach an external archive within the same execution model.

Taken together, these examples illustrate the progressive-disclosure model of VAMOS: a built-in benchmark can be executed in two lines, a custom problem can be defined from a plain callable without subclassing, and the same API extends naturally to fully specified configurations when expert

```python
from vamos import optimize
from vamos.algorithms import NSGAIIConfig

cfg = (NSGAIIConfig.builder()
       .pop_size(100)
       .crossover("sbx", prob=1.0, eta=20)
       .mutation("pm", prob="1/n", eta=20)
       .selection("tournament", size=4)
       .external_archive()
       .build())

result = optimize(
    "dtlz1", algorithm="nsgaii",
    algorithm_config=cfg,
    max_evaluations=50000, seed=42
)
```

Listing 3. Builder API for fine-grained configuration.

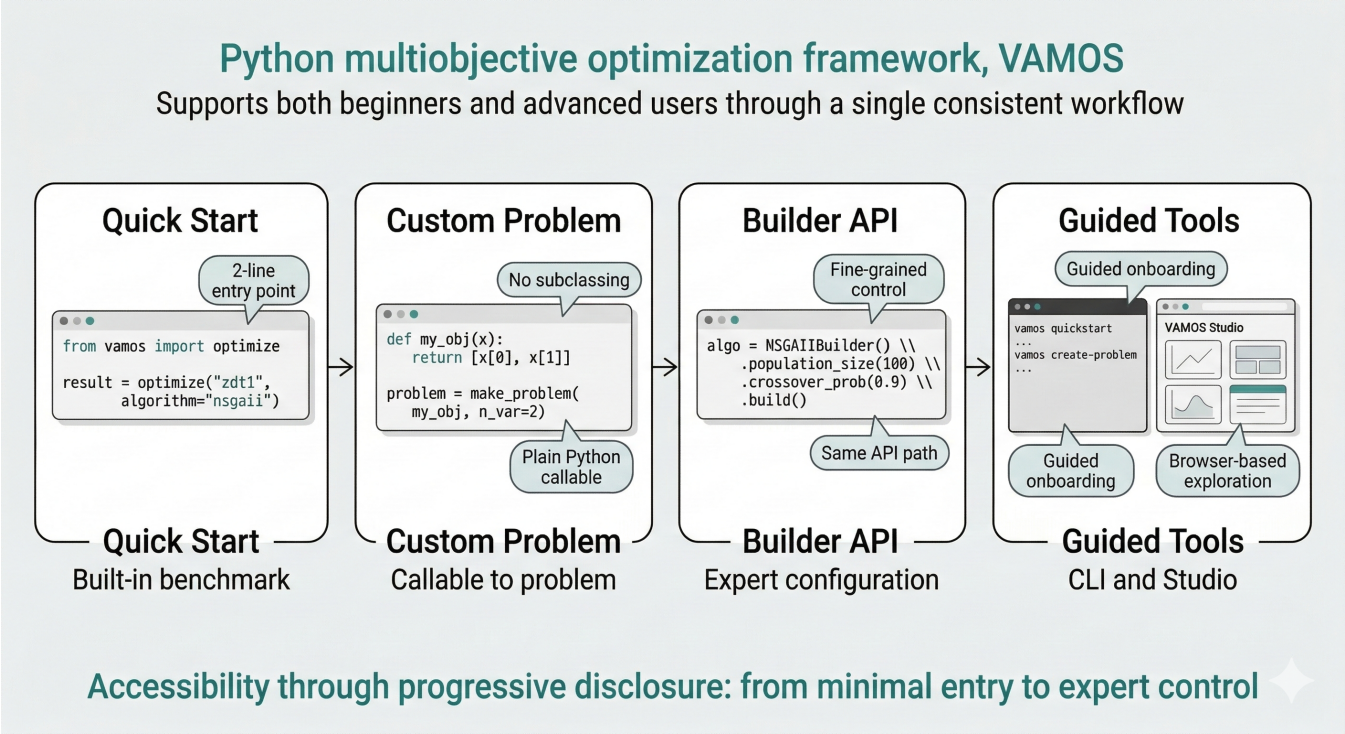


Fig. 6. Accessibility through progressive disclosure in VAMOS. The workflow spans a two-line quick start, callable-based custom problem definition without subclassing, fine-grained configuration through the Builder API, and guided tooling through the command-line interface and VAMOS Studio.

TABLE VIII
ACCESSIBILITY PROXIES FROM CANONICAL ONBOARDING SNIPPETS FOR THE THREE CLOSEST PURE-PYTHON BASELINES CONSIDERED IN THE PAPER. ENTRIES REPORT NON-EMPTY LINES OF CODE WITH IMPORT COUNTS IN PARENTHESES; LOWER IS BETTER. GUIDED TOOLS DENOTES FRAMEWORK-DISTRIBUTED ONBOARDING SUPPORT SUCH AS A GUIDED COMMAND-LINE WIZARD OR GRAPHICAL PROBLEM BUILDER.

| Framework | Built-in Run | Custom Problem | Expert Config | Guided Tools |
|---|---|---|---|---|
| VAMOS | **2 (1)** | **11 (1)** | 16 (2) | ✓ |
| pymoo | 6 (3) | 12 (4) | **13 (5)** | ✗ |
| jMetalPy | 6 (2) | 22 (5) | 17 (5) | ✗ |

Measurement rule: blank lines, comments, plotting, and output-export code are excluded.

control is needed. Beyond the Python API, VAMOS also provides guided CLI entry points (e.g., `vamos quickstart` and `vamos create-problem`) and VAMOS Studio for browser-based onboarding and experiment exploration. The Supplementary Material details the canonical snippets and measurement rules used for the proxy comparison.

We therefore frame accessibility here through objective proxies rather than controlled user studies. A formal usability evaluation with domain-expert participants is left to future work.

## VIII. CONCLUSION

We presented VAMOS, a high-performance Python framework for multi-objective optimization studies that addresses three practical gaps in existing frameworks: (i) a vectorized core with pluggable compute kernels that substantially reduces runtime relative to object-centric libraries, with the largest gains on steady-state and archive-intensive variants and the smallest gaps on SMS-EMOA/WFG and MOEA/D/ZCAT; (ii) rich component-level configurability—with up to 36 parameters, 41 strategies and operators (roughly $3\times$ the variety offered by pymoo), including interchangeable variation operators, configurable offspring batch size for steady-state variants, and optional bounded or unbounded external archives—all exposed as first-class configuration options through a unified builder API; and (iii) an accessible API and guided tooling that lower the entry barrier for domain experts, requiring as few as two lines of code for a complete optimization run while preserving a single path to expert-level configuration. We also release a reproducible cross-framework benchmarking pipeline with semantic alignment and fixed reference Pareto fronts for normalized hypervolume reporting. Under this protocol, paired Wilcoxon signed-rank tests with Holm correction on the aligned NSGA-II comparison confirm that VAMOS achieves statistically equivalent solution quality to state-of-the-art Python frameworks (equivalent on 37/41 NSGA-II problems, non-inferior on 38/41) while substantially reducing runtime. Convergence analysis and Pareto front visualization (Fig. 5) further confirm that these speedups do not degrade convergence behavior. Future work includes:

- **Many-objective benchmarking**: extend the experimental protocol beyond $m = 3$ with scalable indicators.

- **Distributed GPU execution**: explore integration with EvoX [12] for multi-node GPU acceleration.
- **Algorithm configuration**: the current tuning module integrates Optuna, SMAC3, and BOHB as backends (Section VI-B). Future work includes adding interoperability with irace [42], publishing tuning benchmarks that compare the integrated backends on MOEA configuration tasks, and providing richer tuning reports with convergence analysis and statistical summaries.
- **Broader comparisons**: extend the benchmark suite to additional MOEAs and frameworks and include time-to-target and other indicators alongside normalized HV.
- **LLM-assisted experiment planning**: we are exploring an integrated assistant that maps a natural-language problem description to a validated experiment configuration, using an optional LLM provider, to further lower the barrier to entry for non-expert users.
- **User-friendliness evaluation**: conduct controlled user studies with domain experts from fields such as engineering, biology, and operations research to formally assess the usability of the API, the graphical interface, and the documentation, and to identify concrete improvements that further reduce the learning curve for non-specialist users.
- **Alternative encodings**: the current experimental evaluation focuses on continuous optimization; future work will extend benchmarking to the binary, integer, and permutation encodings already supported by VAMOS.
- **Constrained optimization**: evaluate the declarative constraint-handling module on established constrained benchmark suites.

## Data and Code Availability

All data and code supporting the findings of this study (benchmark scripts, reference Pareto fronts, and CSV artifacts used to regenerate tables) are available in the VAMOS repository: https://github.com/vamos-framework/vamos.

## References


[1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, UK: John Wiley & Sons, 2001.

[2] C. A. C. Coello, G. B. Lamont, and D. A. V. Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY, USA: Springer, 2007.

[3] S. Bleuler, M. Laumanns, L. Thiele, and E. Zitzler, "PISA—a platform and programming language independent interface for search algorithms," in *Proc. Evolutionary Multi-Criterion Optimization (EMO)*, ser. Lecture Notes in Computer Science, vol. 2632. Springer, 2003, pp. 494–508.

[4] J. J. Durillo and A. J. Nebro, "jMetal: A Java framework for multi-objective optimization," *Advances in Engineering Software*, vol. 42, no. 10, pp. 760–771, 2011.

[5] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.

[6] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[7] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, and J. Del Ser, "jMetalPy: A Python framework for multi-objective optimization with metaheuristics," *Swarm and Evolutionary Computation*, vol. 51, p. 100598, 2019.

[8] D. Hadka, "Platypus: A free and open source Python library for multiobjective optimization," GitHub repository, https://github.com/Project-Platypus/Platypus, 2015, accessed: 2026.

[9] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.

[10] H. Ishibuchi, L. M. Pang, and K. Shang, "A new framework of evolutionary multi-objective algorithms with an unbounded external archive," in *Proc. 24th European Conf. Artificial Intelligence (ECAI)*. IOS Press, 2020, pp. 283–290.

[11] F. Biscani and D. Izzo, "A parallel global multiobjective framework for optimization: pagmo," *Journal of Open Source Software*, vol. 5, no. 53, p. 2338, 2020.

[12] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, "EvoX: A distributed GPU-accelerated framework for scalable evolutionary computation," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 5, pp. 1649–1662, 2025.

[13] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 102–116, Feb. 2021.

[14] H. Zhao, X.-H. Ning, J.-Y. Li, and J. Liu, "Evolutionary multitask framework with bi-knowledge transfer for multimodal optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 393–407, Feb. 2026.

[15] H. Ye, H. Xu, and C. A. Coello Coello, "Light-EvoOPT: A lightweight evolutionary optimization framework for ultralarge-scale mixed integer linear programs," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 333–347, Feb. 2026.

[16] Z. Ma, Y.-J. Gong, H. Guo, J. Chen, Y. Ma, Z. Cao, and J. Zhang, "LLaMoCo: Instruction tuning of large language models for optimization code generation," *IEEE Transactions on Evolutionary Computation*, 2026, early access.

[17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2019, pp. 2623–2631.

[18] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile Bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.

[19] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 80, 2018, pp. 1437–1446.

[20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Krevelen, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[21] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2015.

[22] M. L.-I. nez *et al.*, "moocore: Multi-objective optimization core library," GitHub repository, https://github.com/multi-objective/moocore, 2024, accessed: 2026.

[23] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evolutionary Computation*, vol. 8, no. 2, pp. 173–195, 2000.

[24] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable multi-objective optimization test problems," in *Proc. Congr. Evol. Comput. (CEC)*, 2002, pp. 825–830.

[25] S. Huband, P. Hingston, L. Barone, and L. While, "A review of multiobjective test problems and a scalable test problem toolkit," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 5, pp. 477–506, 2006.

[26] S. Zapotecas-Martínez, C. A. Coello Coello, H. E. Aguirre, and K. Tanaka, "Challenging test problems for multi- and many-objective optimization," *Swarm and Evolutionary Computation*, vol. 81, p. 101350, 2023.

[27] Q. Zhang, A. Zhou, S. Zhao, P. N. Suganthan, W. Liu, and S. Tiwari, "Multiobjective optimization test instances for the CEC 2009 special session and competition," University of Essex and Nanyang Technological University, Tech. Rep. CEC-MO09, 2008.

[28] R. Tanabe and H. Ishibuchi, "An easy-to-use real-world multi-objective optimization problem suite," *Applied Soft Computing*, vol. 89, p. 106078, 2020.

[29] S. Z. Martínez, A. García-Nájera, and A. Menchaca-Méndez, "Engineering applications of multi-objective evolutionary algorithms: A test suite

of box-constrained real-world problems," *Engineering Applications of Artificial Intelligence*, vol. 122, p. 106192, 2023.

[30] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[31] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[32] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," ETH Zurich, Zurich, Switzerland, Tech. Rep. TIK-Report 103, 2001.

[33] A. Panichella, "An adaptive evolutionary algorithm based on non-Euclidean geometry for many-objective optimization," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*. ACM, 2019, pp. 595–603.

[34] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[35] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "A reference vector guided evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 5, pp. 773–791, 2016.

[36] N. Beume, B. Naujoks, and M. Emmerich, "SMS-EMOA: Multiobjective selection based on dominated hypervolume," *European Journal of Operational Research*, vol. 181, no. 3, pp. 1653–1669, 2007.

[37] E. Zitzler and S. Künzli, "Indicator-based selection in multiobjective search," in *Proc. Int. Conf. Parallel Problem Solving from Nature (PPSN)*. Springer, 2004, pp. 832–842.

[38] HoloViz, "Panel: The powerful data exploration & web app framework for Python," https://panel.holoviz.org, 2024, accessed: 2025-06-01.

[39] J. J. Durillo, A. J. Nebro, F. Luna, and E. Alba, "On the effect of the steady-state selection scheme in multi-objective genetic algorithms," in *Proc. Evolutionary Multi-Criterion Optimization (EMO)*. Springer, 2009, pp. 183–197.

[40] J. Blank, K. Deb, Y. D. Dhebar, S. Bandaru, and H. Seada, "Generating well-spaced points on a unit simplex for evolutionary many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 48–60, 2021.

[41] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, pp. 117–132, 2003.

[42] M. L.-I. nez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle, and M. Birattari, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.